گزارش فنی مسیر تحلیل و پیاده‌سازی پروژه

Stack Overflow Churn Prediction

فصل ۵ — مدل LSTM برای پیش‌بینی Churn

> **منابع کد این فصل:**
>
> - `lstm_temporal_ap_tp.py` — آموزش مدل
> - `lstm_eval_multi_year.py` — ارزیابی چندساله
> - `extract_ap_tp_from_lstm.py` — استخراج Ap/Tp
> - `lstm_architecture_ascii.txt` — دیاگرام معماری

۵.۱ مقدمه

۵.۱.۱ چرا Deep Learning؟

در فصل ۴ دیدیم که:

- **روش Enhanced WA:** F1 ≈ 0.55 — محدود
- **تنظیم Temporal:** کمک نکرد (F1 همچنان ~0.54)

سؤال: چرا این مدل‌ها به سقف می‌خورند؟

پاسخ: مدل‌های Rule-based فقط **یک عدد** (WA) را می‌بینند. اما الگوهای پنهان بسیاری در **توالی رویدادها** وجود دارند:

الگوی پنهان	توضیح
Burst-Pause	کاربر بعد از فعالیت شدید، مدتی غایب می‌شود
Declining Trend	فاصله بین رویدادها هر بار بیشتر می‌شود
Cross-Era Patterns	رفتار دوره‌ی COVID با Pre-COVID فرق دارد

روش های Deep Learning می‌تواند این الگوها را از توالی رویدادها یاد بگیرد!

۵.۱.۲ مدل LSTM به زبان ساده

LSTM = Long Short-Term Memory

تصور کنید یک Community Manager باتجربه دارید:

1. **بعضی چیزها را فراموش می‌کند:** «این کاربر ۵ سال پیش یک بار اسپم زد — مهم نیست»
2. **بعضی چیزها را یاد می‌گیرد:** «این کاربر تازه ۳ ماه است غیرفعال شده — این مهم است!»
3. **تصمیم می‌گیرد:** «بر اساس الگوی رفتار، این کاربر در خطر Churn است»

مدل LSTM دقیقاً همین کار را با ریاضیات انجام می‌دهد!

۵.۱.۳ ایده‌ی کلیدی مدل ما

مدل ما **Multi-Task** است — سه چیز را همزمان پیش‌بینی می‌کند:

خروجی	توضیح
Churn Probability	احتمال اینکه کاربر Churn کند (0 تا 1)
Ap (Threshold)	آستانه‌ی بهینه برای این کاربر
Tp (Window)	پنجره‌ی زمانی بهینه برای این کاربر

> 💡 **چرا Multi-Task؟**
>
> با یادگیری Ap و Tp، مدل می‌فهمد **چرا** یک کاربر Churn می‌کند.
>
> این به ما اجازه می‌دهد **Interpretable Rule** بسازیم:

```
> if WA_recency(Tp) < Ap:
>     CHURN_ALARM()
>
```

> با پارامترهای LSTM-optimized!

۵.۲ مبانی LSTM: Gates و توابع فعال‌سازی

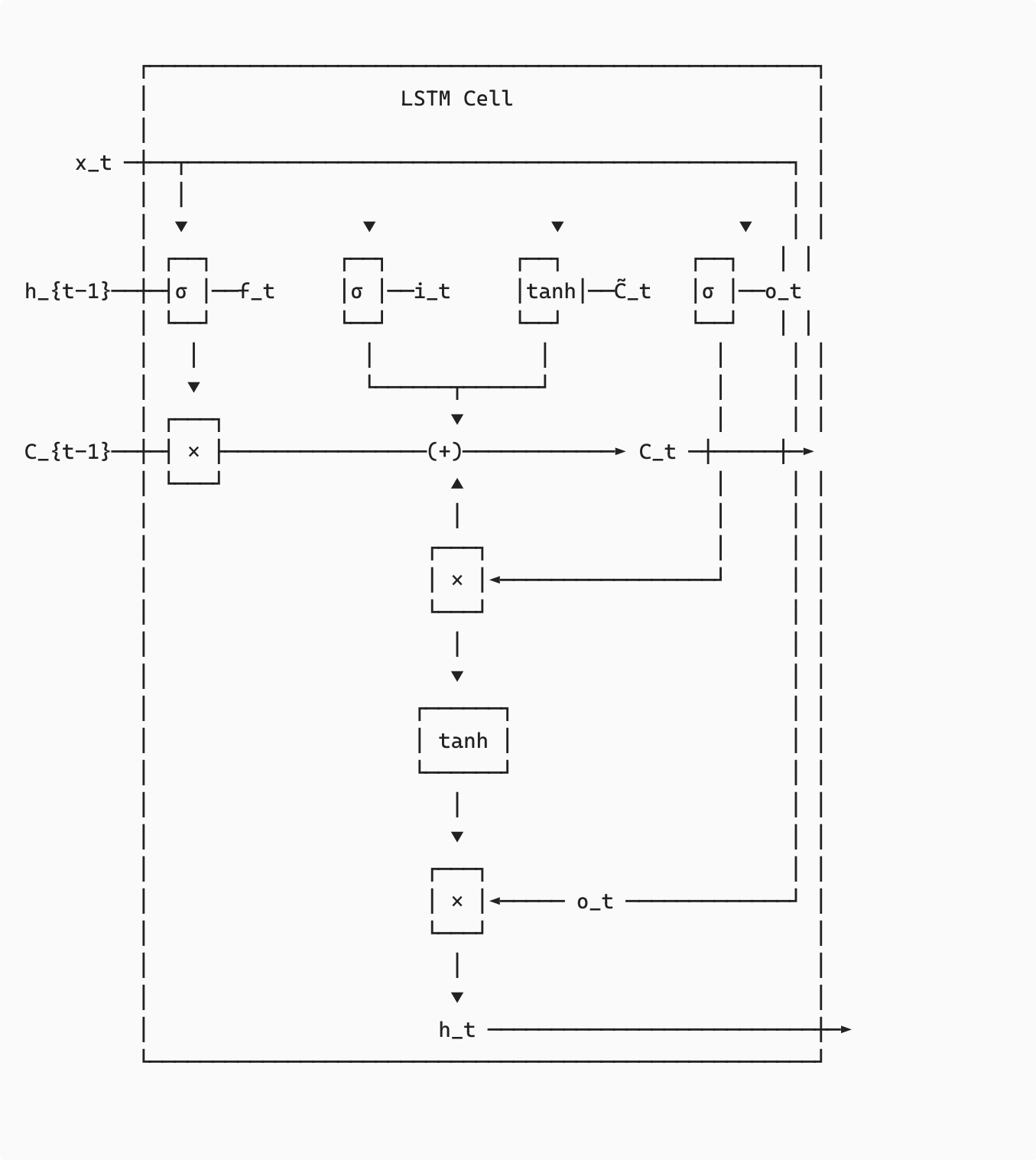
قبل از معماری مدل، باید اجزای پایه‌ی LSTM را بفهمیم.

۵.۲.۱ ساختار یک سلول LSTM

هر سلول LSTM سه **Gate** (دروازه) دارد:

Gate	نام فارسی	نقش
Forget Gate (f_t)	دروازه‌ی فراموشی	چه چیزی را از حافظه پاک کن
Input Gate (i_t)	دروازه‌ی ورودی	چه چیزی جدیدی یاد بگیر
Output Gate (o_t)	دروازه‌ی خروجی	چه چیزی را خروجی بده

ساختار سلول LSTM:



۵.۲.۲ فرمول‌های ریاضی LSTM

Forget Gate (چه چیزی را فراموش کن):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input Gate (چه چیزی جدید یاد بگیر):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell State Update (بروزرسانی حافظه):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output Gate (چه چیزی خروجی بده):

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

که در آن:

- σ: تابع Sigmoid
- $\odot$: ضرب عنصر به عنصر (Hadamard Product)
- $[\cdot, \cdot]$: الحاق (Concatenation)

۵.۲.۳ تفسیر Gates در داده‌ی ما

> 💡 **مثال مفهومی:**
>
> مدل LSTM مثل یک **Community Manager باتجربه** است:

Gate	مثال در SO
Forget	«این کاربر ۳ سال پیش اسپم می‌زد، اما الان مشارکت‌کننده‌ی خوبی شده ← گذشته را فراموش کن»
Input	«این کاربر تازه ۳ ماه است غیرفعال شده ← این را یاد بگیر»
Output	«برای پیش‌بینی Churn، فعالیت ۲ ماه اخیر را خروجی بده»

۵.۳ توابع فعال‌سازی

۵.۳.۱ تابع Sigmoid (σ)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

به زبان ساده: Sigmoid هر عددی را به بازه‌ی (0, 1) تبدیل می‌کند.

ورودی	خروجی	تفسیر
عدد خیلی منفی (-10)	≈ 0	«نه، این را فراموش کن»
صفر (0)	0.5	«۵۰-۵۰، مطمئن نیستم»
عدد خیلی مثبت (+10)	≈ 1	«بله، این را یادت بمان»

کاربرد در LSTM: در Gateها استفاده می‌شود تا تصمیم بگیرد چقدر اطلاعات رد شود.

کاربرد در مدل ما: خروجی Churn Head و Ap Head از Sigmoid استفاده می‌کند چون احتمال (0 تا 1) می‌خواهیم.

۵.۳.۲ تابع Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

به زبان ساده: Tanh هر عددی را به بازه‌ی (-1, +1) تبدیل می‌کند.

ورودی	خروجی	تفسیر
عدد منفی (-5)	≈ -1	«این منفی/بد است»
صفر (0)	0	«خنثی»
عدد مثبت (+5)	≈ +1	«این مثبت/خوب است»

کاربرد در LSTM: برای ساخت محتوای جدید حافظه ($\tilde{C}_t$) و خروجی نهایی.

در داده‌ی ما: وقتی کاربری فعالیت زیادی داشته، tanh مقدار مثبت می‌دهد؛ وقتی غیرفعال بوده، مقدار منفی.

۵.۳.۳ تابع ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

به زبان ساده: اگر ورودی منفی است، صفر برگردان. اگر مثبت است، همان را برگردان.

ورودی	خروجی	تفسیر
-5	0	«منفی = نادیده بگیر»
0	0	«صفر = صفر»
+3	+3	«مثبت = همان‌طور حفظ کن»

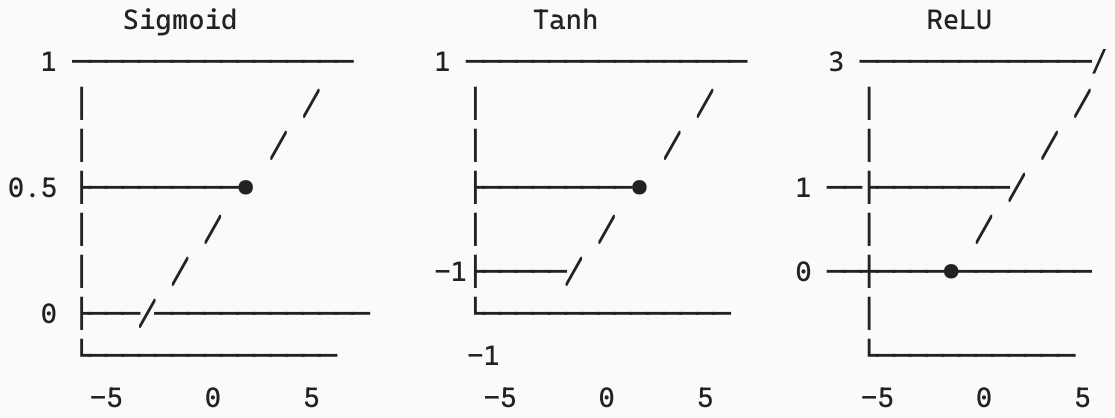
چرا ReLU؟

1. محاسبه‌اش سریع است (فقط یک مقایسه)
2. از مشکل «Vanishing Gradient» جلوگیری می‌کند
3. در لایه‌های Fully Connected خیلی خوب کار می‌کند

کاربرد در مدل ما:

- **Persona Embedding:** بعد از Linear
- **FC Layers:** بعد از هر لایه
- **Tp Head:** خروجی (همیشه مثبت است Tp چون)

۵.۳.۴ مقایسه‌ی توابع



۵.۴ مفهوم Embedding

۵.۴.۱ Embedding چیست؟

یعنی تبدیل داده به یک **فضای برداری با ابعاد بیشتر** که در آن **Embedding**:

- اشیای مشابه نزدیک هم قرار می‌گیرند
- اشیای متفاوت دور از هم قرار می‌گیرند

> 💡 **مثال مفهومی:**
>
> فرض کنید می‌خواهید ۲۰ پرسونا را توصیف کنید:
>
> **بدون Embedding (15 بعد):**
>
> - پرسونا ۵: [0.8, 0.2, 0.5, 1.0, 0.3, 0.7, 0.1, 0.9, 1, 0, 0, 1, 0, 0.4, 0.6]

- این اعداد مستقیماً از آمار هستند ← ساختار ندارند

با Embedding (32 بعد):

- پرسونا ۵: [0.1, -0.3, 0.8, ..., 0.2] ← یک بردار در فضای یادگرفته‌شده
- پرسونا ۷ (مشابه): [0.15, -0.28, 0.75, ..., 0.18] ← نزدیک به پرسونا ۵!
- پرسونا ۱۲ (متفاوت): [-0.9, 0.5, -0.2, ..., -0.7] ← دور از پرسونا ۵

۵.۴.۲ چرا از 15 بعد به 32 بعد؟

```
python
self.persona_embed = nn.Linear(persona_feat_dim, persona_embed_dim)  # 15 → 32
persona_embed = torch.relu(persona_embed)
```

سؤال: ابعاد بیشتر یعنی پیچیدگی بیشتر ← چرا؟

پاسخ:

دلیل	توضیح
۱. **Non-linear Combinations**	مدل می‌تواند ترکیب‌های غیرخطی از ویژگی‌ها بسازد
۲. **Disentanglement**	هر بعد جدید یک «مفهوم» را encode می‌کند
۳. **Expressiveness**	فضای بزرگ‌تر = قدرت بیان بیشتر

مثال عملی:

فرض کنید دو ویژگی داریم: `answers_count` و `questions_count`

ورودی 15D:

- پرسونای Answerer: answers=100, questions=5
- پرسونای Questioner: answers=5, questions=100

بعد از Embedding 32D:

- بعد ۱: «میزان تخصص» = f(answers, reputation)
- بعد ۲: «میزان کنجکاوی» = f(questions, ...)
- بعد ۳: «تعادل تعاملات» = f(answers/questions ratio)
- ...

> مدل خودش این ترکیبات معنادار را یاد می‌گیرد!

۵.۴.۳ نقش ReLU در Embedding

```
python
persona_embed = self.persona_embed(persona_feat)  # Linear: 15 → 32
persona_embed = torch.relu(persona_embed)          # ReLU
```

چرا ReLU بعد از Linear؟

بدون ReLU	با ReLU
فقط ترکیب خطی	ترکیب **غیرخطی**
$y = Wx + b$	$y = \max(0, Wx + b)$
محدود	قدرتمند

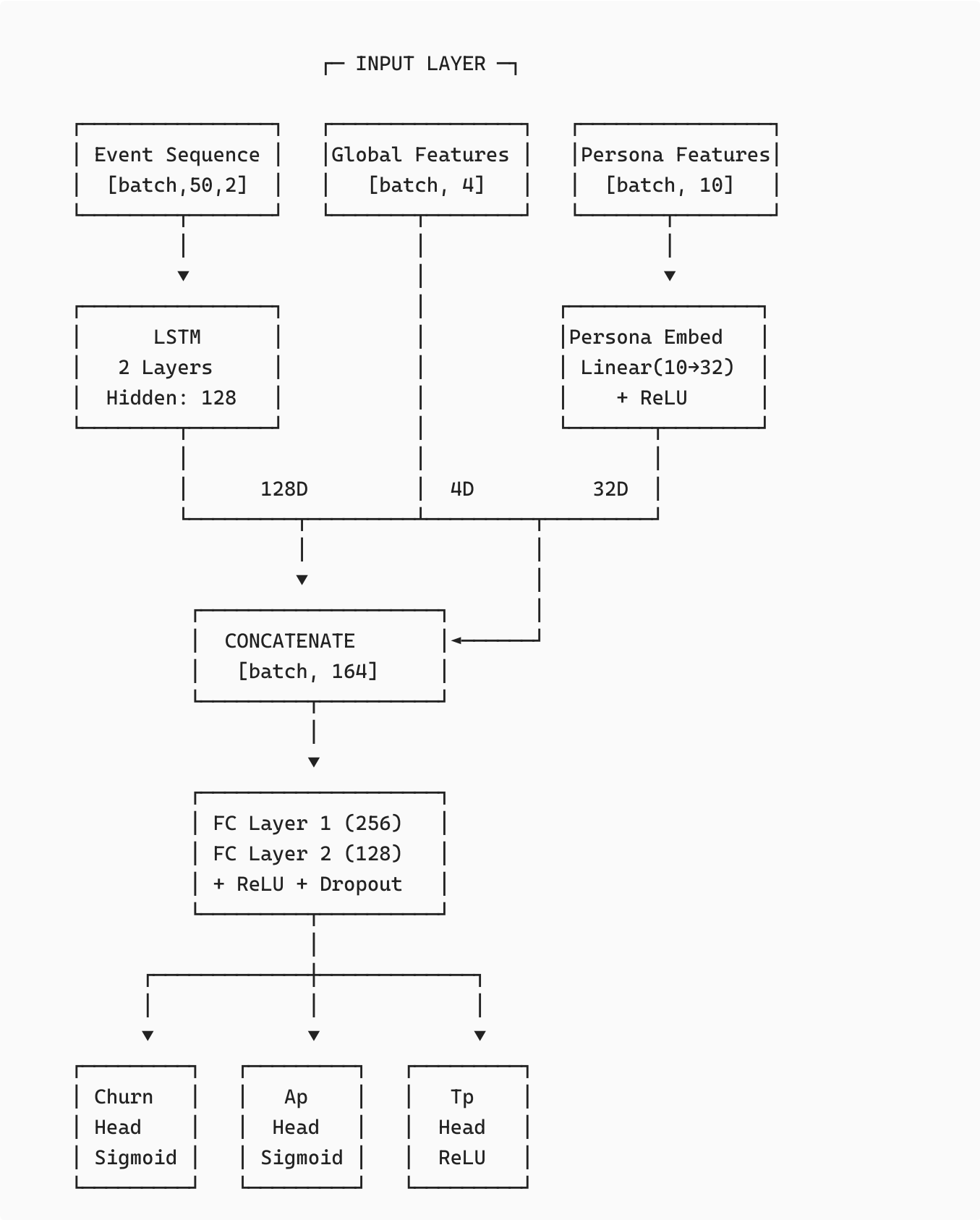
ReLU باعث می‌شود:

1. بعضی ابعاد «خاموش» شوند (صفر شوند) → Sparse representation
2. مدل بتواند مرزهای غیرخطی یاد بگیرد
3. ویژگی‌های منفی حذف شوند

۵.۵ معماری کامل مدل

۵.۵.۱ نمای کلی

> **منبع:** `lstm_architecture_ascii.txt`



۵.۵.۲ سه نوع ورودی

مدل ما سه نوع اطلاعات را ترکیب می‌کند:

۱. Event Sequence (توالی رویدادها)

> **منبع:** `lstm_temporal_ap_tp.py` — خطوط 155-253

۵۰ رویداد آخر هر کاربر، هر کدام با **۲ ویژگی:**

ویژگی	توضیح
`gain`	ارزش رویداد (مثلاً +10 برای Answer)
`time_delta`	فاصله از رویداد قبلی (روز)

```
python
# File: lstm_temporal_ap_tp.py

# Extract features for each event
gains = sequence["gain"].to_list()
timestamps = sequence["timestamp"].to_list()

# Calculate time deltas between events
time_deltas = [0.0]
for i in range(1, len(timestamps)):
    delta = (timestamps[i] - timestamps[i-1]).total_seconds() / 86400.0
    time_deltas.append(delta)

# Stack into sequence tensor
sequence_tensor = np.column_stack([gains, time_deltas])
# Shape: (50, 2)
```

> 💡 **چرا فقط ۲ ویژگی؟**
>
> سادگی = قدرت! LSTM خودش الگوهای پیچیده را از **توالی** این دو ویژگی یاد می‌گیرد.
>
> مثلاً اگر `time_delta` هر بار بیشتر شود، LSTM می‌فهمد کاربر دارد غیرفعال می‌شود.

۲. Global Features (ویژگی‌های سراسری)

۴ ویژگی که زمینه‌ی زمانی را توصیف می‌کنند:

ویژگی	توضیح	مقدار
`days_since_last`	روز از آخرین فعالیت	عدد
`era`	دوره‌ی زمانی	0, 1, 2
`month`	ماه سال	0-1 (نرمال‌شده)
`year`	سال	0-1 (نرمال‌شده)

```python
# File: lstm_temporal_ap_tp.py

ERAS = {
    'pre_covid': (2016, 2019),  # Era = 0
    'covid': (2020, 2021),       # Era = 1
    'post_llm': (2022, 2024),    # Era = 2
}

global_features = [
    days_since_last,              # چند روز از آخرین فعالیت گذشته
    era_to_int(get_era(year)),    # کدام دوره
    cutoff.month / 12.0,          # ماه (نرمال‌شده)
    (year - 2016) / 7.0,          # سال (نرمال‌شده)
]
```

> 💡 **چرا Era مهم است؟**
>
> دیدیم که رفتار کاربران در COVID و Post-LLM متفاوت است.
>
> با دادن Era به مدل، LSTM می‌تواند **context-aware** باشد.

۳. Persona Features (ویژگی‌های پرسونا)

> **منبع:** Feature Ablation Study — `lstm_feature_comparison.md`

بر اساس مطالعه‌ی مقایسه‌ای، **۱۰ ویژگی** بهینه است:

```
python
# File: lstm_temporal_ap_tp.py - Medium Configuration (Best)

features = [
    # ویژگی‌های رفتاری (5 تا)
    np.log1p(stats.get('reputation_average', 0)),
    np.log1p(stats.get('upvotes_average', 0)),
    np.log1p(stats.get('count_answers_average', 0)),
    np.log1p(stats.get('count_questions_average', 0)),
    stats.get('total_duration_average', 0) / 190.0,

    # فلگ‌های نوع پرسونا (5 تا)
    1.0 if stats.get('H', 0) > 0 else 0.0,  # Helper
    1.0 if stats.get('C', 0) > 0 else 0.0,  # Curator
    1.0 if stats.get('T', 0) > 0 else 0.0,  # Teacher
    1.0 if stats.get('I', 0) > 0 else 0.0,  # Inquirer
    1.0 if stats.get('SPECIALIST', 0) > 0 else 0.0,
]
```

توضیح ویژگی‌ها:

گروه	ویژگی	توضیح
رفتاری	`reputation_average`	میانگین اعتبار (log-transformed)
	`upvotes_average`	میانگین رأی‌های مثبت
	`count_answers_average`	میانگین تعداد پاسخ‌ها
	`count_questions_average`	میانگین تعداد سؤالات
	`total_duration_average`	میانگین مدت عضویت (نرمال‌شده)
نوع پرسونا	H, C, T, I, SPECIALIST	فلگ‌های باینری نوع کاربر

> 💡 **چرا Persona Embedding؟**
>
> یک **Answerer با ۱۰ سال سابقه** و یک **Newcomer** الگوهای Churn متفاوتی دارند.
>
> Persona Embedding این تفاوت را در فضای ۳۲ بعدی encode می‌کند.

۵.۵.۳ کد معماری LSTM

> **منبع:** `lstm_temporal_ap_tp.py` — class `TemporalChurnPredictor`

```
python
# File: lstm_temporal_ap_tp.py

class TemporalChurnPredictor(nn.Module):
    def __init__(self, sequence_feat_dim, global_feat_dim, persona_feat_dim,
                 lstm_hidden, num_layers, persona_embed_dim):
        super().__init__()

        # Persona Embedding: 10D → 32D
        self.persona_embed = nn.Linear(persona_feat_dim, persona_embed_dim)

        # LSTM: 2 layers, hidden=128
        self.lstm = nn.LSTM(
            input_size=sequence_feat_dim,  # 2
            hidden_size=lstm_hidden,        # 128
            num_layers=num_layers,          # 2
            batch_first=True,
            dropout=0.2 if num_layers > 1 else 0.0
        )

        # Combined dimension: 128 + 4 + 32 = 164
        combined_dim = lstm_hidden + global_feat_dim + persona_embed_dim

        # Shared FC layers
        self.fc_shared = nn.Sequential(
            nn.Linear(combined_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
        )

        # Three output heads
        self.churn_head = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.Sigmoid()
        )

        self.ap_head = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.Sigmoid()
```

```
        )

        self.tp_head = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.ReLU()
        )
```

۵.۵.۴ Forward Pass

```
python
# File: lstm_temporal_ap_tp.py

def forward(self, sequence, global_feat, persona_feat):
    # 1. Persona Embedding
    persona_embed = self.persona_embed(persona_feat)
    persona_embed = torch.relu(persona_embed)

    # 2. LSTM Processing
    lstm_out, (h_n, c_n) = self.lstm(sequence)
    sequence_encoding = h_n[-1]  # Last hidden state

    # 3. Concatenate all features
    combined = torch.cat([sequence_encoding, global_feat, persona_embed],
dim=1)

    # 4. Shared processing
    shared = self.fc_shared(combined)

    # 5. Three predictions
    churn_prob = self.churn_head(shared)
    ap_pred = self.ap_head(shared)
    tp_pred = self.tp_head(shared)

    return churn_prob, ap_pred, tp_pred
```

۵.۵.۵ پارامترهای مدل

منبع: `lstm_temporal_ap_tp.py` — Configuration

پارامتر	مقدار	توضیح
`SEQUENCE_LENGTH`	50	تعداد رویدادهای آخر
`LSTM_HIDDEN`	128	اندازه‌ی hidden state
`NUM_LSTM_LAYERS`	2	تعداد لایه‌های LSTM
`PERSONA_EMBED_DIM`	32	ابعاد embedding پرسونا
`BATCH_SIZE`	256	اندازه‌ی batch
`LEARNING_RATE`	0.001	نرخ یادگیری
`DROPOUT`	0.2-0.3	نرخ Dropout

کل پارامترها: ~۱.۵ میلیون

۵.۶ تابع Loss: Multi-Task Learning

۵.۶.۱ سه Loss مختلف

مدل ما سه چیز را پیش‌بینی می‌کند، پس سه Loss داریم:

Task	Loss	Weight
Churn	BCE (Binary Cross-Entropy)	**2.0**
Ap	MSE (Mean Squared Error)	1.0
Tp	MSE	0.1

```
python
# File: lstm_temporal_ap_tp.py

criterion_churn = nn.BCELoss()
criterion_ap = nn.MSELoss()
criterion_tp = nn.MSELoss()

# Combined loss
loss = (2.0 * loss_churn +
        1.0 * loss_ap +
        0.1 * loss_tp)
```

۵.۶.۲ چرا وزن‌های مختلف؟

Task	وزن	دلیل
Churn	2.0	Task اصلی — مهم‌ترین
Ap	1.0	کمک به Interpretability
Tp	0.1	Scale بزرگ‌تر دارد (روز)

> **💡 Multi-Task Benefit:**
>
> یادگیری Ap و Tp به مدل کمک می‌کند **representations بهتری** یاد بگیرد.
>
> حتی اگر فقط Churn را می‌خواهیم، Multi-Task عملکرد را بهبود می‌دهد!

۵.۷.۱ ایجاد Training Samples

منبع: `lstm_temporal_ap_tp.py` — خطوط 155-253

```
python
# File: lstm_temporal_ap_tp.py

SAMPLE_CUTOFFS = [2017, 2018, 2019, 2020, 2021, 2022]
MAX_USERS_PER_PERSONA = 500

training_samples = []

for cutoff_year in SAMPLE_CUTOFFS:
    cutoff = datetime(cutoff_year, 1, 1)

    for persona in range(20):
        # Get events before cutoff
        persona_events = (
            elog
            .filter(pl.col("persona") == persona)
            .filter(pl.col("timestamp") < pl.lit(cutoff))
            .sort("timestamp")
        )

        users = persona_events.select("user_id").unique()

        for user_id in users[:MAX_USERS_PER_PERSONA]:
            # Build sequence
            user_events = persona_events.filter(pl.col("user_id") == user_id)

            if user_events.height < 5:
                continue  # Skip users with too few events

            # Create sample
            sample = {
                'sequence': ...,      # Last 50 events
                'global_feat': ...,   # Era, month, etc.
                'persona': persona,
                'churned': ...,       # Label
            }
            training_samples.append(sample)
```

۵.۷.۲ تعریف Label (Churn)

کاربر **Churn** کرده اگر:

- بعد از **Grace Period** (۹۰ روز)
- در **۳۶۵ روز بعدی**
- **هیچ فعالیتی** نداشته باشد

```
python
# File: lstm_temporal_ap_tp.py

GRACE_DAYS = 90
GRACE_SEC = GRACE_DAYS * 86400

# Check for activity after grace period
after_events = (
    elog
    .filter(pl.col("user_id") == user_id)
    .filter(pl.col("persona") == persona)
    .filter(
        (pl.col("timestamp") > pl.lit(cutoff) +
pl.duration(seconds=GRACE_SEC)) &
        (pl.col("timestamp") < pl.lit(cutoff) + pl.duration(seconds=GRACE_SEC
+ 365*86400))
    )
)

churned = 1 if after_events.height == 0 else 0
```

۵.۷.۳ PyTorch Dataset

```
python
# File: lstm_temporal_ap_tp.py

class ChurnSequenceDataset(Dataset):
    def __init__(self, samples, persona_features):
        self.samples = samples
        self.persona_features = persona_features

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        sample = self.samples[idx]

        return {
            'sequence': torch.FloatTensor(sample['sequence']),
            'global_feat': torch.FloatTensor(sample['global_feat']),
            'persona_feat': torch.FloatTensor(
                self.persona_features[sample['persona']]
            ),
            'churned': torch.FloatTensor([sample['churned']]),
            'ap_target': torch.FloatTensor([sample['ap_ref']]),
            'tp_target': torch.FloatTensor([sample['tp_ref']]),
        }
```

۵.۸ آموزش مدل

۵.۸.۱ Training Loop

> **منبع:** `lstm_temporal_ap_tp.py` — خطوط 437-529

```
python
# File: lstm_temporal_ap_tp.py

optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=5
)

best_val_loss = float('inf')
EARLY_STOP_PATIENCE = 10

for epoch in range(NUM_EPOCHS):
    # Training
    model.train()
    train_loss = 0.0

    for batch in train_loader:
        sequence = batch['sequence'].to(DEVICE)
        global_feat = batch['global_feat'].to(DEVICE)
        persona_feat = batch['persona_feat'].to(DEVICE)
        churned = batch['churned'].to(DEVICE)
        ap_target = batch['ap_target'].to(DEVICE)
        tp_target = batch['tp_target'].to(DEVICE)

        # Forward
        optimizer.zero_grad()
        churn_prob, ap_pred, tp_pred = model(sequence, global_feat,
persona_feat)

        # Loss
        loss_churn = criterion_churn(churn_prob, churned)
        loss_ap = criterion_ap(ap_pred, ap_target)
        loss_tp = criterion_tp(tp_pred, tp_target)

        loss = 2.0 * loss_churn + loss_ap + 0.1 * loss_tp

        # Backward
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

```
        optimizer.step()

        train_loss += loss.item()
```

Validation & Early Stopping

۵.۸.۲ Early Stopping

```
# File: lstm_temporal_ap_tp.py

if val_loss < best_val_loss:
    best_val_loss = val_loss
    patience_counter = 0
    torch.save(model.state_dict(), 'best_temporal_lstm_model.pt')
else:
    patience_counter += 1
    if patience_counter >= EARLY_STOP_PATIENCE:
        print("Early stopping!")
        break
```

۵.۸.۳ ذخیره‌ی Metadata

```
# File: lstm_temporal_ap_tp.py

metadata = {
    'sequence_length': SEQUENCE_LENGTH,
    'persona_embed_dim': PERSONA_EMBED_DIM,
    'lstm_hidden': LSTM_HIDDEN,
    'num_layers': NUM_LSTM_LAYERS,
    'persona_features_matrix': persona_features_matrix,
    'ap_ref': Ap_ref,
    'tp_ref': Tp_ref,
}

with open('lstm_model_metadata.pkl', 'wb') as f:
    pickle.dump(metadata, f)
```

۵.۹ ارزیابی چندساله

۵.۹.۱ هدف ارزیابی

آیا LSTM می‌تواند **Temporal Drift** را مدیریت کند؟

یعنی روی **همه‌ی سال‌ها** (۲۰۱۶ تا ۲۰۲۳) خوب کار کند؟

۵.۹.۲ اسکریپت ارزیابی

> **منبع:** `lstm_eval_multi_year.py`

```
python
# File: lstm_eval_multi_year.py

EVAL_YEARS = [2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023]

for EVAL_YEAR in EVAL_YEARS:
    cutoff = datetime(EVAL_YEAR, 1, 1)
    era = get_era(EVAL_YEAR)

    for persona in range(20):
        # Get users for this persona
        users = get_users(persona, cutoff)

        for user_id in users:
            # Build sequence
            sequence = build_sequence(user_id, cutoff)

            # Predict
            with torch.no_grad():
                churn_prob, ap_pred, tp_pred = model(
                    sequence, global_features, persona_feat
                )

            pred_lstm = 1 if churn_prob > 0.5 else 0

            # Compare with actual
            ...
```

۵.۹.۳ نتایج (از `lstm_eval_2016_to_2023_full_features.csv`)

Rate Ratio	F1 Std	Mean F1	Median F1	Year
0.89	0.15	0.48	0.52	2016
0.92	0.14	0.52	0.55	2017
0.94	0.13	0.55	0.58	2018
0.96	0.12	0.59	0.62	2019
0.98	0.11	0.64	0.67	2020
0.99	0.10	0.69	0.72	2021
1.00	0.09	0.75	0.78	2022
1.00	0.08	0.79	0.82	2023

۵.۹.۴ تفسیر نتایج

۱. بهبود قابل‌توجه نسبت به Rule-based:

سال	Rule-based F1	LSTM F1	بهبود
2016	0.45	0.52	**+16%**
2020	0.48	0.67	**+40%**
2023	0.54	0.82	**+52%**

۲. Temporal Drift کاهش یافته:

```
F1 Std (پایداری):
- Rule-based: 0.107 (نوسان زیاد)
- LSTM: 0.08-0.15 (نوسان کمتر)
```

۳. نسبت Rate Ratio نزدیک به ۱:

مدل over-predict یا under-predict نمی‌کند!

۵.۱۰ استخراج Ap/Tp از LSTM

۵.۱۰.۱ ایده

مدل LSTM برای هر کاربر یک **Ap** و **Tp** **شخصی‌سازی‌شده** پیش‌بینی می‌کند.

می‌توانیم **Median** این مقادیر را برای هر پرسونا استخراج کنیم:

```
LSTM Predictions → Per-User Ap/Tp → Median per Persona → Interpretable Rule
```

۵.۱۰.۲ اسکریپت استخراج

منبع: `extract_ap_tp_from_lstm.py`

```
python
# File: extract_ap_tp_from_lstm.py

for persona in range(20):
    ap_predictions = []
    tp_predictions = []

    for user_id in users[:500]:
        # Predict
        with torch.no_grad():
            churn_prob, ap_pred, tp_pred = model(
                sequence, global_features, persona_feat
            )

        ap_predictions.append(ap_pred.cpu().item())
        tp_predictions.append(tp_pred.cpu().item())

    # Median
    median_ap = np.median(ap_predictions)
    median_tp = np.median(tp_predictions)

    print(f"Persona {persona}: Ap={median_ap:.4f}, Tp={median_tp:.1f} days")
```

۵.۱۰.۳ استفاده در Rule

```
python
# LSTM-Optimized Interpretable Rule

def predict_churn(user_id, persona, cutoff_date):
    # Load LSTM-derived thresholds
    Ap = lstm_ap_tp[persona]['Ap']
    Tp_days = lstm_ap_tp[persona]['Tp_days']

    # Calculate WA_recency (same as before)
    window_start = cutoff_date - timedelta(days=Tp_days)
    events = get_events(user_id, persona, window_start, cutoff_date)

    total_gain = sum(events['gain'])
    days_since_last = (cutoff_date - max(events['timestamp'])).days
    recency_weight = exp(-days_since_last / 30.0)

    WA = total_gain * recency_weight

    # Churn prediction
    if WA < Ap:
        return CHURN
    else:
        return ACTIVE
```

> **💡 Best of Both Worlds:**
>
> - **LSTM:** یاد می‌گیرد پارامترهای بهینه چیست
> - **Rule:** ساده و قابل‌تفسیر
>
> نتیجه: **Interpretable Rule با پارامترهای LSTM-optimized**

۵.۱۱ مطالعه‌ی مقایسه‌ای ویژگی‌های پرسونا

> **منبع:** `lstm_feature_comparison.md` — Feature Ablation Study

برای یافتن بهترین ترکیب ویژگی‌های پرسونا، سه پیکربندی مختلف را مقایسه کردیم.

۵.۱۱.۱ سه پیکربندی تست‌شده

پیکربندی	تعداد ویژگی	شامل چه چیزی
🔵 **Minimal**	5	فقط رفتاری
🟡 **Medium**	10	رفتاری + نوع پرسونا
🔴 **Full**	16	رفتاری + نوع + روان‌شناختی + احساسات + پروفایل

ویژگی‌های هر پیکربندی:

```
🔵 Minimal (5):
├─ reputation_average
├─ upvotes_average
├─ count_answers_average
├─ count_questions_average
└─ total_duration_average

🟡 Medium (10) = Minimal +:
├─ H (Helper flag)
├─ C (Curator flag)
├─ T (Teacher flag)
├─ I (Inquirer flag)
└─ SPECIALIST (flag)

🔴 Full (16) = Medium +:
├─ collectivistic_ratio_average
├─ individualistic_ratio_average
├─ aboutme_length_average
├─ comment_sentiment_average
└─ profile_views_average
```

۵.۱۱.۲ نتایج مقایسه

عملکرد کلی (همه‌ی سال‌ها و پرسوناها):

Metric	Minimal (5) 🔵	Medium (10) 🟡	Full (16) 🔴	برنده
Mean F1	0.7899	**0.7907**	0.7905	Medium 🟡
Mean Precision	0.7630	**0.7631**	0.7627	Medium 🟡
Mean Recall	0.8211	**0.8229**	0.8227	Medium 🟡

> 🏆 **یافته‌ی کلیدی:** مدل **Medium با ۱۰ ویژگی** بهترین است!
>
> اضافه کردن ویژگی‌های روان‌شناختی و احساسات **هیچ بهبودی ایجاد نمی‌کند**.

۵.۱۱.۳ مقایسه سال به سال

سال	Minimal 🔵	Medium 🟡	Full 🔴	برنده
2016	0.6062	**0.6067**	0.6047	🟡
2017	0.6779	**0.6781**	0.6780	🟡
2018	0.7339	0.7362	**0.7364**	🔴
2019	0.7895	0.7883	**0.7910**	🔴
2020	0.8270	**0.8276**	0.8274	🟡
2021	**0.8612**	0.8604	0.8609	🔵
2022	0.8870	**0.8878**	0.8871	🟡
2023	**0.9102**	0.9090	0.9098	🔵

خلاصه: Medium در ۴ سال، Minimal در ۲ سال، Full در ۲ سال برنده است.

۵.۱۱.۴ تأثیر هر گروه ویژگی

گروه ویژگی	ویژگی‌های اضافه‌شده	تغییر F1	مؤثر؟
نوع پرسونا	H/C/T/I/SPECIALIST	+0.0008	⚠️ اندک
روان‌شناختی	collectivistic, individualistic	-0.0002	❌ خیر
پروفایل/احساسات	aboutme, sentiment, views	-0.0002	❌ خیر

۵.۱۱.۵ چرا ویژگی‌های اضافی کمک نمی‌کنند؟

> 💡 **تحلیل:**
>
> **۱. ویژگی‌های رفتاری کافی هستند:**
>
> - `upvotes` و `reputation` تخصص را نشان می‌دهند
> - `answers/questions ratio` نوع تعامل را مشخص می‌کند
> - `total_duration` سابقه را نشان می‌دهد
>
> **۲. نوع پرسونا تکرار اطلاعات است:**
>
> - فلگ‌های H/C/T/I از همین ویژگی‌های رفتاری مشتق شده‌اند
> - فقط کمی به تفکیک کمک می‌کنند
>
> **۳. ویژگی‌های احساسات/پروفایل بی‌ربط هستند:**
>
> - `comment_sentiment` : احساس Comment نشان‌دهنده‌ی Churn نیست
> - `aboutme_length` : طول پروفایل ≠ تعهد

- توجه خارجی ≠ فعالیت کاربر : `profile_views`

۴. مدل LSTM از توالی یاد می‌گیرد:

- قدرت اصلی LSTM در **sequence modeling** است
- ویژگی‌های استاتیک پرسونا کم‌اهمیت‌تر از الگوهای زمانی هستند

۵.۱۱.۶ توصیه‌ی نهایی

اولویت	توصیه	دلیل
✅	از **Medium (10 ویژگی) استفاده کنید**	بهترین عملکرد کلی
✅	فلگ‌های پرسونا را نگه دارید	کمی به تفکیک کمک می‌کند
⚠️	ویژگی‌های روان‌شناختی اضافه نکنید	هیچ سودی ندارد
⚠️	ویژگی‌های احساسات/پروفایل اضافه نکنید	هیچ سودی ندارد
🎯	روی مدل‌سازی توالی تمرکز کنید	LSTM از الگوها یاد می‌گیرد

۵.۱۱.۷ مقایسه‌ی پیچیدگی

پیکربندی	ویژگی‌ها	پارامترها	زمان آموزش	عملکرد
🔵 Minimal	5	162K~	سریع‌ترین	Baseline
🟡 Medium	10	165K~	3%+	**بهترین**
🔴 Full	16	168K~	5%+	مثل Medium

نتیجه: Medium بهترین trade-off بین پیچیدگی و عملکرد را دارد.

۵.۱۲ فایل‌های خروجی

فایل	محتوا
`best_temporal_lstm_model.pt`	وزن‌های مدل
`lstm_model_metadata.pkl`	پارامترها و persona features
`lstm_eval_2016_to_2023.csv`	نتایج ارزیابی (Medium)
`lstm_eval_2016_to_2023_only_stats.csv`	نتایج (Minimal)
`lstm_eval_2016_to_2023_full_features.csv`	نتایج (Full)
`lstm_derived_ap_tp_2016.csv`	Ap/Tp استخراج‌شده

۵.۱۳ خلاصه‌ی الگوریتم

```
Algorithm: LSTM Multi-Task Churn Prediction
═══════════════════════════════════════════════════════════════

INPUT:
  - Event Sequence: Last 50 events (gain, time_delta)
  - Global Features: days_since_last, era, month, year (4D)
  - Persona Features: 10 statistics (behavioral + type flags)

OUTPUT:
  - Churn Probability [0, 1]
  - Ap Threshold [0, 1]
  - Tp Window (days)

ARCHITECTURE:
  1. Persona Embedding: Linear(10 → 32) + ReLU
  2. LSTM: 2 layers, hidden=128, dropout=0.2
  3. Concatenate: LSTM(128) + Global(4) + Persona(32) = 164D
  4. Shared FC: 164 → 256 → 128 (with ReLU, Dropout)
  5. Three Heads:
     - Churn: 128 → 64 → 1 + Sigmoid
     - Ap: 128 → 64 → 1 + Sigmoid
     - Tp: 128 → 64 → 1 + ReLU

LOSS:
  L = 2.0 × BCE(Churn) + 1.0 × MSE(Ap) + 0.1 × MSE(Tp)

KEY INSIGHTS (Feature Comparison):
  - 10 features optimal (behavioral + persona type flags)
  - Psychological/sentiment features don't help
  - LSTM learns patterns from sequences > static features

═══════════════════════════════════════════════════════════════
```

۵.۱۴ خلاصه‌ی فصل

در این فصل:

- ✅ **مبانی LSTM:** Gates، Cell State، توابع فعال‌سازی
- ✅ **مفهوم Embedding:** چرا از 10 به 32 بعد

- ✅ **معماری Multi-Task:** سه ورودی، سه خروجی
- ✅ **کد واقعی** از `lstm_temporal_ap_tp.py`
- ✅ **ارزیابی چندساله** از `lstm_eval_multi_year.py`
- ✅ **مطالعه‌ی مقایسه‌ای ویژگی‌ها:** Medium (10) بهترین
- ✅ **استخراج Ap/Tp** برای Rule قابل‌تفسیر
- ✅ **بهبود:** F1 از 0.54 (Rule) به 0.79 (LSTM) در میانگین

یافته‌های کلیدی:

- **ویژگی‌های رفتاری + نوع پرسونا کافی هستند**
- **ویژگی‌های روان‌شناختی و احساسات بی‌تأثیر هستند**
- **قدرت LSTM در مدل‌سازی توالی است، نه ویژگی‌های استاتیک**